从零开始的 LLM Scaling Law 探索：小规模实验 Demo

肖添宁

2026 年 6 月

摘要

本文构建了一套单卡 H100 上可复现的小规模 LLM scaling law demo，用于探索模型参数量 N、训练 token 数 D、训练计算量 C 与 validation loss 之间的关系。实验基于 GPT-2 tokenizer 和 RoPE decoder-only GPT，使用约 1.03B tokens 的 mixed 数据以及六类 source 数据，并引入 doc-level gzip complexity 与文档内重复度指标描述数据分布。通过 fixed-step 学习曲线、fixed-FLOPs IsoFLOP U 型曲线和 compute-optimal frontier 拟合，本文在 10^{16} 至 10^{18} FLOPs 范围内得到 mixed 数据上的近似关系

$$N^\star \propto C^{0.407}, \qquad D^\star \propto C^{0.593}.$$

进一步的 source 实验显示，不同数据分布对应的最优模型–数据配比存在差异。本文结果仅作为一个完整的研究 demo，展示从数据处理、训练记录、复杂度分析到 scaling 拟合与报告生成的实验闭环。

目录

1 引言 **2**

2 定义与数量关系 **2**

2.1 模型参数量 N 4

2.2 训练计算量 C 4

2.3 训练 token 数 D 与损失函数 $\mathcal{L}$ 5

2.4 给定算力 C 下的最优解 7

3 研究方案 **7**

3.1 硬件条件与训练设定 7

3.2 数据集处理与复杂度度量 7

3.3 超参数的选取 9

4 实验结果 **10**

4.1 Mixed fixed-step：训练动力学 10

4.2 Mixed IsoFLOP：固定算力下的 U 型曲线 11

4.3 数据复杂度与 source 固定步长实验 12

4.4 Source IsoFLOP：数据分布改变最优配比 14

5 总结与讨论 **16**

1 引言

自 Kaplan 等人 [1] 在 2020 年系统研究语言模型的 scaling law 以来，损失函数 $\mathcal{L}$ 与模型参数量 N、训练 token 数 D 和训练计算量 C 之间的幂律关系逐渐成为理解大模型训练过程的重要经验规律：

$$\mathcal{L}(N, D) \sim AN^{-\alpha} + BD^{-\beta}, \qquad \text{s.t.} \quad C \sim ND. \tag{1}$$

在其研究的模型族和数据规模范围内，固定计算预算下的最优策略更偏向于增大模型规模，并且模型不必训练到完全收敛。随后，Henighan 等人进一步指出类似规律不仅出现在语言模型，也可扩展到图像、视频和数学推理等自回归生成任务 [2]。2022 年，Hoffmann 等人提出的 Chinchilla scaling law 重新审视了固定算力下模型参数量和训练 token 数的最优配比，指出许多早期大模型实际上存在 undertrained 的问题，即参数量过大而训练 token 数不足 [3]。此后，GPT-4 Technical Report 中的 predictable scaling 思想也说明，小规模实验如果设计得足够干净，可以为更大规模训练提供可解释的预测依据 [4]。

不过，scaling law 相关研究仍然存在许多争议和没有解决的问题。具体而言，我们希望回答：在给定算力预算下，什么时候应该增大模型参数量，什么时候应该增加训练 token 数，如何衡量训练数据的质量，以及如何判断继续训练的边际收益是否已经下降。进一步地，除了训练成本，推理成本也应被纳入考虑 [5]。对实际研究和工程而言，更重要的问题包括：给定少量 GPU 时，如何设计可信的小规模实验；如何定义参数量 N 和训练计算量 C；如何避免 batch size、learning rate、tokenizer、数据重复等因素影响 scaling 结论；如何度量数据复杂度；以及如何判断继续训练的边际收益是否下降。

本文作为个人初步探索 scaling law 的小规模 demo，在单张 H100 的算力条件下，参考 Karpathy 的开源项目 nanoGPT [6]，搭建并运行了一组小规模实验。下面我们从第一性原理出发，首先明确基本变量的定义。

2 定义与数量关系

常见的 decoder-only Transformer 架构，可以参数化为

$$(L, d_{\text{model}}, h, V, M),$$

其中 L 是 Transformer block 层数，d_{model} 是 hidden dimension，h 是 attention head 数，V 是词表大小，M 是设定的最大上下文长度。实验统一使用 GPT-2 tokenizer，因此 $V = 50257$；并且上下文长度取 $M = 1024$；按照常见的工程经验，FFN hidden size 取 $4d_{\text{model}}$。

为了用一个整数参数 N_{depth} 控制模型规模，本文采用 nanochat [7] 当中的参数化控制方案，这里固定了 attention head dimension $d_h = 128$：

$$L = N_{\text{depth}}, \quad d_{\text{model}} = \left\lceil \frac{64 \cdot N_{\text{depth}}}{128} \right\rceil \cdot 128, \quad h = d_{\text{model}}/128. \tag{2}$$

这样只改变 N_{depth} 就能得到结构相似且参数量单调增加的一组模型。具体的对应关系见表 1。

表 1: 不同深度 N_{depth} 对应的模型参数大小

N_{depth}	L	d_{model}	h	M	V	Block params	Vocab params	Total params	N
1	1	128	1	1024	50257	196,608	6,432,896	6,629,504	6.630M
2	2	128	1	1024	50257	393,216	6,432,896	6,826,112	6.826M
3	3	256	2	1024	50257	2,359,296	12,865,792	15,225,088	15.225M
4	4	256	2	1024	50257	3,145,728	12,865,792	16,011,520	16.012M
5	5	384	3	1024	50257	8,847,360	19,298,688	28,146,048	28.146M
6	6	384	3	1024	50257	10,616,832	19,298,688	29,915,520	29.916M
7	7	512	4	1024	50257	22,020,096	25,731,584	47,751,680	47.752M
8	8	512	4	1024	50257	25,165,824	25,731,584	50,897,408	50.897M
9	9	640	5	1024	50257	44,236,800	32,164,480	76,401,280	76.401M
10	10	640	5	1024	50257	49,152,000	32,164,480	81,316,480	81.316M
11	11	768	6	1024	50257	77,856,768	38,597,376	116,454,144	116.454M
12	12	768	6	1024	50257	84,934,656	38,597,376	123,532,032	123.532M
13	13	896	7	1024	50257	125,239,296	45,030,272	170,269,568	170.270M
14	14	896	7	1024	50257	134,873,088	45,030,272	179,903,360	179.903M
15	15	1024	8	1024	50257	188,743,680	51,463,168	240,206,848	240.207M
16	16	1024	8	1024	50257	201,326,592	51,463,168	252,789,760	252.790M
17	17	1152	9	1024	50257	270,729,216	57,896,064	328,625,280	328.625M
18	18	1152	9	1024	50257	286,654,464	57,896,064	344,550,528	344.551M
19	19	1280	10	1024	50257	373,555,200	64,328,960	437,884,160	437.884M
20	20	1280	10	1024	50257	393,216,000	64,328,960	457,544,960	457.545M

2.1 模型参数量 N

一般来说，总参数量可以写为

$$N = N_{\text{emb}} + L\left(N_{\text{att}} + N_{\text{ffn}}\right) + 2d_{\text{model}} + d_{\text{model}}V, \tag{3}$$

其中 N_{emb} 包含 token embedding 和 positional embedding，N_{att} 是每层 self-attention 参数，N_{ffn} 是每层 FFN 参数，$2d_{\text{model}}$ 来自 final LayerNorm，$d_{\text{model}}V$ 是 LM head。若考虑 weight tying，即 LM head 与 token embedding 参数共享，则不考虑最后一项的 $d_{\text{model}}V$.

下面详细展开，若位置编码使用 learned absolute positional embedding，则

$$N_{\text{emb}} = \underbrace{d_{\text{model}}V}_{\text{token embedding}} + \underbrace{d_{\text{model}}M}_{\text{positional embedding}}. \tag{4}$$

可以预见，对于小模型，位置编码的权重会占较大比例，因此本文使用不需要学习参数的 RoPE，其通过旋转变换将相对位置信息注入 attention。从而 $N_{\text{emb}} = d_{\text{model}}V$.

对于 self-attention 每层主要包含 Q, K, V projection 和 output projection。考虑到现在主流的 LLM 都不引入线性层的 bias，则

$$N_{\text{att}} = \underbrace{3d_{\text{model}}^2}_{Q,K,V} + \underbrace{d_{\text{model}}^2}_{\text{output projection}} + \underbrace{2d_{\text{model}}}_{\text{LayerNorm}} = 4d_{\text{model}}^2 + 2d_{\text{model}}. \tag{5}$$

根据工程经验，FFN 常使用 $4d_{\text{model}}$ hidden size，因此每层 FFN 参数约为

$$N_{\text{ffn}} = \underbrace{d_{\text{model}} \cdot 4d_{\text{model}}}_{\text{up projection}} + \underbrace{4d_{\text{model}} \cdot d_{\text{model}}}_{\text{down projection}} + \underbrace{2d_{\text{model}}}_{\text{LayerNorm}} = 8d_{\text{model}}^2 + 2d_{\text{model}}. \tag{6}$$

若忽略 $\mathcal{O}(d_{\text{model}})$ 阶小量后，L 层 Block 当中的参数为有

$$L\left(N_{\text{att}} + N_{\text{ffn}}\right) \approx 12Ld_{\text{model}}^2. \tag{7}$$

由于本文使用的框架采用了 weight tying 和 RoPE，从而总参数量为

$$N = d_{\text{model}}V + L\left(N_{\text{att}} + N_{\text{ffn}}\right) + 2d_{\text{model}} \approx d_{\text{model}}V + 12Ld_{\text{model}}^2. \tag{8}$$

此外，我们采用 GPT-2 的 tokenizer，因此 $V = 50257$.

2.2 训练计算量 C

训练计算量常用 FLOPs 衡量。一次训练 step 包含 forward 和 backward；经验上 backward 计算量约为 forward 的两倍，因此

$$C = C_{\text{forward}} + C_{\text{backward}} \approx 3C_{\text{forward}}. \tag{9}$$

对于矩阵乘法 $C_{ij} = A_{ik}B_{kj}$，计算量约为 $2ijk$ FLOPs。一次 forward 的主要计算可以写为

$$C_{\text{forward}} = C_{\text{emb}} + L(C_{\text{att}} + C_{\text{ffn}}) + C_{\text{logits}} \approx L(C_{\text{att}} + C_{\text{ffn}}). \tag{10}$$

embedding 查表计算较小可忽略；logits projection 在大词表下并不总能忽略，但在常见 scaling law 粗略估计中，主项集中在 Transformer block 的 dense matrix 上。

设序列长度为 S，attention 中需要经历的计算量为

$$C_{\text{att}} = \underbrace{3(2d_{\text{model}}^2 S)}_{Q,K,V\ \text{linear}} + \underbrace{2Sd_{\text{model}}S}_{QK^\top} + \underbrace{2hS^2}_{\text{scale / mask}} + \underbrace{5hS^2}_{\text{softmax}} + \underbrace{2S^2 d_{\text{model}}}_{\text{Attention}\times V} + \underbrace{2Sd_{\text{model}}^2}_{\text{output projection}} \tag{11}$$
$$= 8Sd_{\text{model}}^2 + 4S^2 d_{\text{model}} + 7S^2 h.$$

对于前馈网络 FFN

$$C_{\text{ffn}} = \underbrace{2S(d_{\text{model}} \cdot 4d_{\text{model}})}_{\text{up projection}} + \underbrace{2S(4d_{\text{model}} \cdot d_{\text{model}})}_{\text{down projection}} = 16Sd_{\text{model}}^2. \tag{12}$$

因此每层的计算量为

$$24Sd_{\text{model}}^2 + 4S^2 d_{\text{model}} + 7S^2 h = 24Sd_{\text{model}}^2 \left[1 + \frac{S}{6d_{\text{model}}} + \frac{7hS}{24d_{\text{model}}^2}\right]. \tag{13}$$

考虑到 $h = d_{\text{model}}/d_h$ 我们得到 L 层的计算量为

$$24LSd_{\text{model}}^2 \left[1 + \frac{S}{6d_{\text{model}}} \left(1 + \frac{7}{4d_h}\right)\right]. \tag{14}$$

再结合公式(7)，如果仅仅考虑领头项，那么训练一次长度为 S 的序列的 forward 约为 $2SN$, 从而再考虑上 backward 需要约 $6SN$。如果训练共看到 D 个 token，则得到常见近似 $C \approx 6ND$。

但是值得注意的是，上述 $C \approx 6ND$ 是一种主项近似。它隐含了非 embedding 参数占主导、logits projection 可忽略，以及 attention 二次项相对较小等条件。对于小模型、大词表或长上下文设置，$4S^2 d_{\text{model}}$ 与 C_{logits} 可能不再是小修正，此时应回到更完整的 FLOPs 表达式进行估算。然而，本项目暂时为了方便起见，就直接用 $C \approx 6ND$ 来估算了。此外，nanochat 项目当中考虑到了第二项的影响，即 $4S^2 d_{\text{model}}$ 对计算量的贡献，从而有 $C \approx 6ND + 12LSDd_{\text{model}}$.

2.3 训练 token 数 D 与损失函数 $\mathcal{L}$

训练量 D 表示模型训练过程中实际看到的 token 数。若 micro batch size 为 B，序列长度为 S，gradient accumulation steps 为 G，训练 step 数为 T，则

$$D = BSGT. \tag{15}$$

本文固定

$$B = 16, \qquad S = 1024, \qquad G = 8,$$

因此

$$D = 131072\,T. \tag{16}$$

需要区分 unique corpus tokens 和 training tokens seen。前者是训练集实际包含的 token stream 规模；后者是训练过程中模型累计读取的 token 数。如果 D 超过某个数据源的 unique tokens，顺序读取器会 wrap，并记录 epochs seen。

此外，我们将数据集按 $98\%, 1\%, 1\%$ 的比例分为三份: 数据集，验证集 (validation set)，测试集。分别用于训练使用、在训练过程中评估训练效果、训练结束后评估训练效果。

给定一段长度为 S 的 token 序列

$$(x_1, x_2, \ldots, x_S),$$

decoder-only Transformer 的训练目标是 next-token prediction。也就是说，在位置 t，模型只能利用前缀

$$x_{\le t} = (x_1, \dots, x_t)$$

来预测下一个 token x_{t+1}。由于采用 causal mask，位置 t 不能看到未来的 token。单条序列上的负对数似然损失为

$$\mathcal{L}(\theta; x_{1:S}) = -\frac{1}{S-1} \sum_{t=1}^{S-1} \log p_\theta(x_{t+1} \mid x_{\le t}). \tag{17}$$

实际训练时，一个 batch 中包含 B 条长度为 S 的序列，并在 gradient accumulation 后更新一次参数。因此 train loss 是当前训练 mini-batch 上 next-token cross entropy 的平均值：

$$\mathcal{L}_{\text{train}} = -\frac{1}{B(S-1)} \sum_{b=1}^{B} \sum_{t=1}^{S-1} \log p_\theta(x_{t+1}^{(b)} \mid x_{\le t}^{(b)}). \tag{18}$$

它反映的是模型在当前训练数据片段上的拟合情况，因此波动会受到 batch 内容、数据顺序和优化状态影响。

为了评估模型是否真正学到了可泛化的语言规律，本文使用 validation set 计算 validation loss。验证时模型不进行反向传播，也不更新参数，只在验证集 token stream 上计算同样的 next-token cross entropy。若第 j 个验证 batch 的损失为

$$\ell_j(\theta) = -\frac{1}{B(S-1)} \sum_{b=1}^{B} \sum_{t=1}^{S-1} \log p_\theta(y_{t+1}^{(b)} \mid y_{\le t}^{(b)}), \tag{19}$$

则使用 m 个验证 batch 得到的 validation loss 定义为

$$\mathcal{L}_{\text{val}}^{(m)} = \frac{1}{m} \sum_{j=1}^{m} \ell_j(\theta). \tag{20}$$

本文同时记录三类损失。第一类是 train loss，用于观察优化过程是否正常下降。第二类是 quick validation loss，记为

$$\mathcal{L}_{\text{val}}^{\text{qui}},$$

每 10 step 评估一次，本文中使用较少验证 batch，因此时间分辨率高，但统计噪声更大。第三类是 full validation loss，记为

$$\mathcal{L}_{\text{val}}^{\text{full}},$$

每 500 step 以及训练结束时评估一次，使用更多验证 batch，因此更稳定，作为本文 IsoFLOP 曲线和 scaling exponent 拟合的主指标。

需要强调的是，quick validation loss 主要用于展示训练动力学，例如不同模型规模在训练早期的下降速度；而 full validation loss 更接近我们关心的泛化误差。因此，在固定算力预算下比较不同模型大小时，本文使用 full validation loss，而不是 train loss 或 quick validation loss。

为了进一步降低单个 checkpoint 的统计涨落，本文在 scaling 拟合时使用尾段平滑验证损失。设某个 run 的最后若干个 full validation checkpoints 构成集合 $\mathcal{T}_{\text{tail}}$，则定义

$$\mathcal{L}_{\text{tail}} = \frac{1}{|\mathcal{T}_{\text{tail}}|} \sum_{t \in \mathcal{T}_{\text{tail}}} \mathcal{L}_{\text{val},t}^{\text{full}}. \tag{21}$$

当 full validation checkpoints 数量不足时，本文退化使用 final full validation loss。这样做的目的是避免某一次 final evaluation 的偶然波动直接影响 fixed-FLOPs 下最优模型规模 $N^\star$ 的判断。

2.4 给定算力 C 下的最优解

通常将 validation loss 写为如下形式

$$\mathcal{L}(N, D) = E + AN^{-\alpha} + BD^{-\beta}. \tag{22}$$

在固定算力预算下，通常有约束形式：

$$C = \kappa ND. \tag{23}$$

因此问题变为：在所有满足 $\kappa ND = C$ 的模型–数据组合中，寻找使 validation loss 最小的 (N, D)。

构造拉格朗日函数

$$\mathcal{J}(N, D, \lambda) = E + AN^{-\alpha} + BD^{-\beta} + \lambda(\kappa ND - C). \tag{24}$$

最优点满足

$$\frac{\partial \mathcal{J}}{\partial N} = 0, \qquad \frac{\partial \mathcal{J}}{\partial D} = 0, \qquad \frac{\partial \mathcal{J}}{\partial \lambda} = 0.$$

由前两个条件消去 λ，可得

$$A\alpha N^{-\alpha} = B\beta D^{-\beta}. \tag{25}$$

这个关系表示：在最优配比处，继续增大模型规模与继续增加训练 token 数的边际收益应当平衡。进一步结合 $\kappa ND = C$，得到

$$N^\star \propto C^{\frac{\beta}{\alpha+\beta}}, \qquad D^\star \propto C^{\frac{\alpha}{\alpha+\beta}}, \qquad \mathcal{L}^\star - E \propto C^{-\frac{\alpha\beta}{\alpha+\beta}}. \tag{26}$$

3 研究方案

3.1 硬件条件与训练设定

所有实验在单张 NVIDIA H100 80GB HBM3 上完成，最大 fixed-FLOPs budget 到 10^{18} FLOPs。训练使用 bfloat16 autocast，优化器状态保持 FP32；使用 PyTorch SDPA、`torch.compile`、fused AdamW 和 TF32。训练阶段不使用 KV cache。

模型设计上，本文选择 RoPE 而不是 learned positional embedding，从而避免位置编码参数进入 N_{total}；使用 LayerNorm 而不是 RMSNorm，尽管 RMSNorm 在当前大模型中更常见；不使用 Dropout，以减少小规模 scaling 实验中的随机正则化变量。优化器统一使用 AdamW。

为了避免 batch size 与 learning rate 影响 scaling 结论，本文固定 global batch tokens：

$$B_{\text{tok}} = 16 \times 1024 \times 8 = 131072.$$

学习率固定为 peak learning rate 3×10^{-4}，warmup 后 cosine decay 到 3×10^{-5}。

3.2 数据集处理与复杂度度量

本文使用六类英文数据：story、code、dialogue、math、news 和 encyclopedia。数据处理流程包括清洗、去重、train/validation/test 划分、固定 seed 的 doc shuffle，以及 GPT-2 tokenizer 编码。最终构造两种训练模式：

- mixed stream：六类数据合并成约 1.03B train tokens，用于主 scaling 实验；

- source stream：每类数据单独构建约 170M train tokens，用于观察数据分布差异。

训练时采用 doc shuffle 后的顺序读取，而不是每次从二进制文件中随机抽取片段。这样可以更清楚地记录模型实际看到的数据量和 wrap 次数，避免小数据集在随机采样中出现不可控重复。

数据复杂度方面，本文对每个文档计算 raw text gzip 与 token-id gzip 两类指标。设第 i 篇文档为 x_i，GPT-2 tokenizer 后得到 token 序列

$$\tau_i = (\tau_{i,1}, \tau_{i,2}, \ldots, \tau_{i,n_i}),$$

其中 n_i 是该文档的 GPT-2 token 数。raw gzip 在原始 UTF-8 文本上计算：

$$\mathrm{RawBPT}_i = \frac{8\,|\mathrm{gzip}(\mathrm{utf8}(x_i))|_{\mathrm{byte}}}{n_i}. \tag{27}$$

token-id gzip 则先将 GPT-2 token id 序列写成定长二进制表示后再压缩：

$$\mathrm{TokenBPT}_i = \frac{8\,|\mathrm{gzip}(\mathrm{bytes}(\tau_i))|_{\mathrm{byte}}}{n_i}. \tag{28}$$

因此表 2 中的 Raw bpt 和 Token bpt 分别是 RawBPT_i 与 $\mathrm{TokenBPT}_i$ 在对应数据源上的文档级统计摘要；图 4 则把每篇文档的 $(\mathrm{RawBPT}_i, \mathrm{TokenBPT}_i)$ 作为二维点云展示。

这里没有直接使用普通压缩率

$$r_i = \frac{|\mathrm{gzip}(\mathrm{utf8}(x_i))|_{\mathrm{byte}}}{|\mathrm{utf8}(x_i)|_{\mathrm{byte}}}, \tag{29}$$

原因是压缩率的分母是原始 byte 数，而语言模型训练和 loss 统计的基本单位是 token。二者关系为

$$\mathrm{RawBPT}_i = 8 r_i \frac{|\mathrm{utf8}(x_i)|_{\mathrm{byte}}}{n_i}. \tag{30}$$

也就是说，bits-per-token 在普通压缩率之外进一步校正了每个 token 对应的原文字节数，使复杂度指标和 language modeling 的 per-token loss 更一致。这种方法参考 gzip complexity 相关思路 [8]，可作为廉价的数据复杂度估计。然而，gzip 复杂度不等价于数据质量；它更适合作为静态可压缩性指标，最终仍应结合 validation loss、loss 下降速度、ROI 和数据重复情况一起判断。

除 gzip 复杂度外，本文还记录了一个简单的文档内重复度指标 Duplicate score。对第 i 篇文档，将文本转为小写后用正则表达式提取词集合

$$\mathcal{W}_i = (w_{i,1}, w_{i,2}, \ldots, w_{i,m_i}),$$

其中 m_i 是该文档中词的总数。定义

$$\mathrm{Dup}_i = 1 - \frac{|\mathrm{unique}(\mathcal{W}_i)|}{|\mathcal{W}_i|}. \tag{31}$$

当一篇文档中几乎没有重复词时，Dup_i 接近 0；当少数词或符号模式反复出现时，Dup_i 增大。需要注意的是，该指标衡量的是单篇文档内部的词级重复程度，而不是跨文档的去重率。表 2 中的 Duplicate score 是对应数据源上 Dup_i 的文档级平均值。

表 2: 六类训练数据的规模与文档级复杂度统计。

数据源	Train docs	Train tokens	Raw bpt	Token bpt	Duplicate score
story	782,898	174.30M	17.43	14.03	0.482
code	42,822	170.31M	6.47	6.44	0.626
dialogue	421,050	172.71M	17.94	13.25	0.463
math	81,986	171.00M	12.65	10.62	0.589
news	301,131	172.50M	17.72	12.85	0.459
encyclopedia	161,428	170.44M	16.46	12.55	0.493

3.3 超参数的选取

本文包含三类主要实验：

1. **Fixed-step mixed**：在 mixed 1B 数据上训练 8 个模型，N_{depth} 为 $1, 3, 5, 7, 9, 11, 13, 15$，每个模型训练 1e4 step，用于展示训练动力学。

2. **Fixed-step source**：六类 source 分别训练，N_{depth} 为 $1, 5, 9, 13$，每个模型训练 1e3 step，用于比较不同数据分布下的学习难度。

3. **IsoFLOP**：在固定算力预算下扫描不同模型规模。mixed 实验使用 $C \in \{10^{16}, 3\times10^{16}, 10^{17}, 3\times10^{17}, 10^{18}\}$；source 实验选择 story、code、encyclopedia 三类代表数据源做 refined 对比。

   IsoFLOP 曲线的拟合方式为：对每个固定 C，分别在 $\log_{10} N$ 与 $\log_{10} D$ 轴上拟合二次函数

$$\mathcal{L} = ax^2 + bx + c. \tag{32}$$

若 $a > 0$ 且 vertex 落在观测范围内，则认为该预算下找到了 observed optimum。随后用所有 observed optima 拟合

$$N^\star \propto C^{\frac{\beta}{\alpha+\beta}}, \qquad D^\star \propto C^{\frac{\alpha}{\alpha+\beta}}.$$

需要说明的是，六类 source 当中存在一定的数据 wrap。最严重的情况出现在 code 和 encyclopedia 的 $C = 10^{17}$、depth 1 实验中：训练 tokens 约为 2.514B，而单个 source 的可用 tokens 只有约 170M，因此 epochs seen 达到约 14.7。除这两个最严重的点外，其余已完成 source 补点的 wrap 数均不超过 6；若进一步排除 depth 3 的两个点，则其余点的 wrap 数均不超过 4。

4 实验结果

4.1 Mixed fixed-step：训练动力学

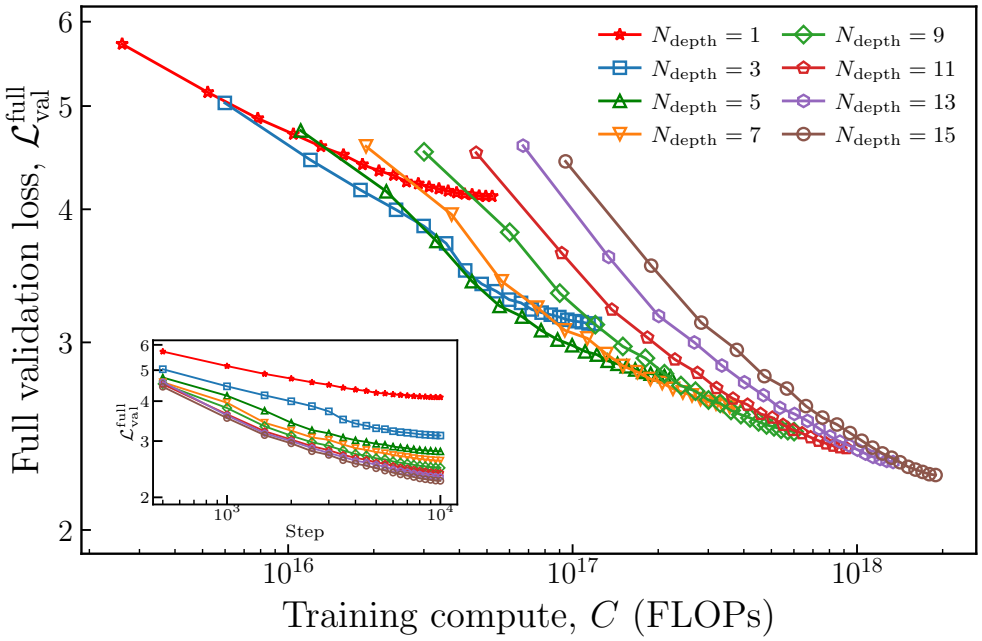


图 1: Mixed 数据上的 fixed-step 学习曲线。主图横轴为训练过程中累计消耗的计算量 C，纵轴为 full validation loss；左下角 inset 展示同一批实验在训练 step 轴上的变化。step 视角反映相同训练步数下不同模型规模的学习动力学，而 FLOPs 视角进一步体现不同模型单步计算成本的差异。

首先考虑六类数据混合后的 mixed stream，并固定每个模型训练的 step 数为 1e4。图 1 展示了不同模型规模下 full validation loss 随累计训练计算量 C 的变化。左下角 inset 则展示同一批实验在 step 轴上的变化，用于区分“训练步数视角”和“计算量视角”。

从 step 视角看，模型规模增大通常会带来更快的 loss 下降和更低的最终 validation loss。这说明在固定 batch size、固定训练步数和相同优化条件下，增大模型容量能够提升 mixed 数据上的拟合能力和泛化表现。尤其是在训练后期，小模型的 loss 下降逐渐变慢，而中大模型仍然保持更明显的下降趋势。

然而，当横轴换成累计训练计算量 C 后，不同模型之间的关系不再只是简单的“模型越大越好”。大模型每一步消耗的 FLOPs 更多，因此其曲线起点更靠右；小模型虽然单步计算成本更低，可以在较小计算量下完成更多参数更新，但后期容易进入收益递减区域。

总的来说，图 1 提供了后续 IsoFLOP 实验的指导：若想研究 compute-optimal scaling，不能只比较相同 step 下的最终 loss，而需要在固定计算预算 C 下同时改变模型参数量 N 与训练 token 数 D。在固定 C 的条件下，较大的模型意味着较少的训练 token，较小的模型意味着更多的训练 token，二者之间存在 trade-off。因此，下一步需要构造 fixed-FLOPs 的 IsoFLOP 曲线，直接寻找每个计算预算下使 validation loss 最低的模型规模。

4.2 Mixed IsoFLOP：固定算力下的 U 型曲线

在 fixed-step 实验之后，本文进一步进行 mixed 数据上的 fixed-FLOPs IsoFLOP 实验。与 fixed-step 实验不同，IsoFLOP 实验固定训练计算预算 C，并扫描不同模型规模 N。在给定 C 和 N 后，训练 token 数由

$$D \approx \frac{C}{6N} \tag{33}$$

确定。因此，同一条 IsoFLOP 曲线上的每个点具有近似相同的训练计算量，但模型参数量和训练 token 数不同：较小模型可以看到更多 token，较大模型则每个 token 的训练计算成本更高，只能看到更少 token。

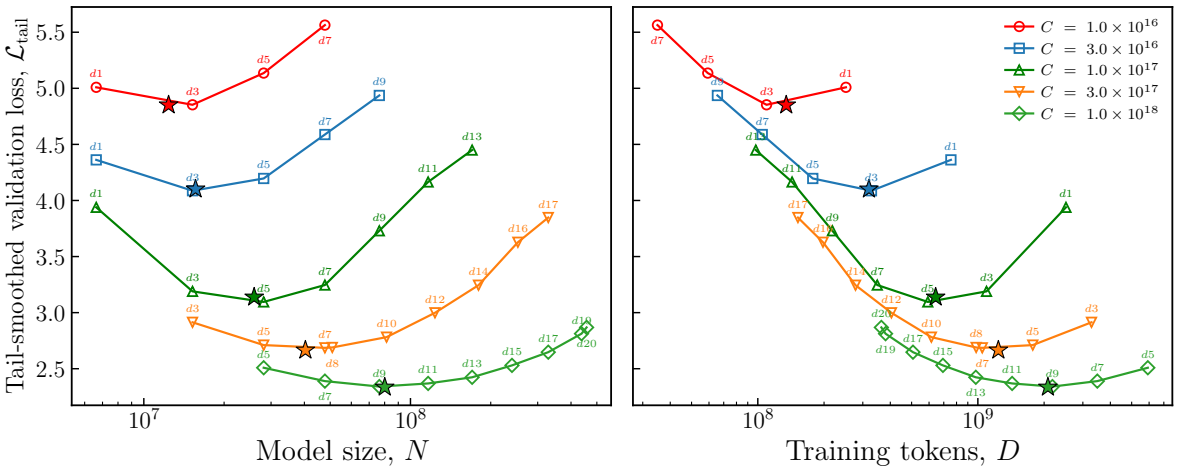


图 2: Mixed 数据上的 fixed-FLOPs IsoFLOP 曲线。左图展示尾段平滑验证损失 $\mathcal{L}_{\text{tail}}$ 关于模型参数量 N 的变化，右图展示同一批实验关于训练 token 数 D 的变化。每条曲线对应一个固定计算预算 C，星号表示在 log-space 二次拟合下得到的 observed optimum。

图 2 展示了 mixed 数据上不同计算预算下的 IsoFLOP 曲线。左图以模型参数量 N 为横轴，右图以训练 token 数 D 为横轴，纵轴均为尾段平滑后的 full validation loss，即 $\mathcal{L}_{\text{tail}}$。可以看到，在固定 C 下，validation loss 关于 N 呈现较清楚的 U 型趋势：当模型过小时，模型容量不足，即使能训练较多 token，最终 loss 仍然较高；当模型过大时，给定算力下可训练 token 数不足，模型训练不充分，validation loss 也会升高。U 型曲线的最低点对应当前计算预算下的 compute-optimal 模型规模 $N^\star$。

为了从离散实验点中估计最优点，本文对每个固定计算预算分别拟合

$$\mathcal{L}_{\text{tail}}(\log_{10} N) = a_N(\log_{10} N)^2 + b_N \log_{10} N + c_N, \tag{34}$$

并取二次函数顶点作为 $N^\star$。若 $a_N > 0$ 且顶点落在实际扫描到的模型参数范围内，则将该 budget 记为 observed optimum。类似地，本文也直接对 token 数做二次拟合，并得到 $D^\star$。这里的 $D^\star$ 是直接由 $\mathcal{L}_{\text{tail}}$–$\log D$ 曲线拟合得到，而不是简单由 $D = C/(6N^\star)$ 反推得到。

本实验在

$$C \in \{10^{16}, 3\times10^{16}, 10^{17}, 3\times10^{17}, 10^{18}\}$$

五个计算预算上都得到了 observed-range 的二次拟合最低点。拟合得到的 mixed compute-optimal frontier 如表 3 所示。

表 3: Mixed 数据上由 IsoFLOP 二次拟合得到的 compute-optimal frontier。$N^\star$ 来自 $\mathcal{L}_{\text{tail}}(\log N)$ 的二次拟合，$D^\star$ 来自 $\mathcal{L}_{\text{tail}}(\log D)$ 的直接二次拟合。

C (FLOPs)	$N^\star$	$D^\star$	$\mathcal{L}^\star_{\text{tail}}$
1.0×10^{16}	12.39M	134.62M	4.851
3.0×10^{16}	15.63M	319.94M	4.101
1.0×10^{17}	25.93M	642.93M	3.138
3.0×10^{17}	40.36M	1.239B	2.665
1.0×10^{18}	80.08M	2.081B	2.335

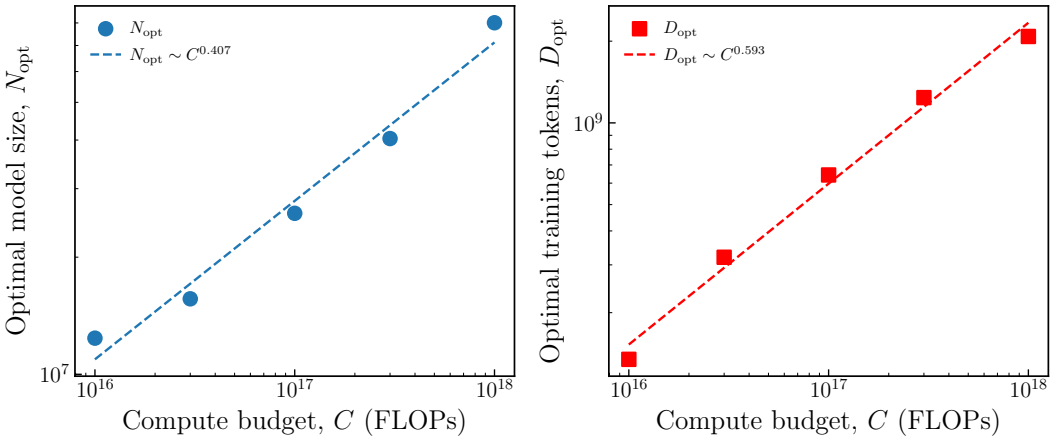


图 3: Mixed 数据上的 compute-optimal frontier。每个点来自图 2 中对应 fixed-FLOPs 曲线的二次拟合最低点；虚线为 log-log 空间中的幂律拟合。结果给出 $N^\star \propto C^{0.407(35)}$ 和 $D^\star \propto C^{0.593(35)}$。

进一步地，图 3 将每个计算预算下的最优点画在 log-log 坐标中，并拟合 compute-optimal scaling 关系，得到

$$N^\star = 28(2)\,\text{M}\left(\frac{C}{10^{17}}\right)^{0.407(35)}, \qquad D^\star = 597(34)\,\text{M}\left(\frac{C}{10^{17}}\right)^{0.593(35)}. \tag{35}$$

从数值上看，参数量的指数小于数据集的指数，表示随着计算预算增加，最优策略更偏向于增加训练 token 数，而不是等比例增加模型参数量。这与 Chinchilla-style scaling 中“模型大小和数据量需要共同增长”的结论方向一致，但具体指数并不应被解释为通用大模型 scaling law。此外，

$$\frac{D^\star}{N^\star} \approx 21\,\text{M}\left(\frac{C}{10^{17}}\right)^{0.185}$$

数据集 token 数与参数量基本接近 $20:1$ 的配比。

4.3 数据复杂度与 source 固定步长实验

在 mixed 数据实验之外，本文进一步比较六类数据源本身的复杂度差异和学习动力学差异。图 4 展示了每篇训练文档在

$$(\text{RawBPT}, \text{TokenBPT})$$

二维空间中的分布。横轴 RawBPT 表示原始 UTF-8 文本经 gzip 压缩后的 bits per GPT-2 token，纵轴 TokenBPT 表示 GPT-2 token id 序列写成定长二进制后经 gzip 压缩得到的 bits per GPT-2

token。图中淡色点表示采样文档，黑边大点表示每类数据源的 median，椭圆表示可视区域内的一倍协方差范围。

从图中可以看到，不同数据源在 gzip complexity 空间中并不完全重合。code 的 RawBPT 和 TokenBPT 都明显较低，说明其局部格式结构更强、可压缩性更高；math 也具有较强的模板和符号结构，但相对 code 更分散。story、dialogue、news 和 encyclopedia 主要分布在更高的复杂度区域，其中 story 的 token-id complexity 偏高，说明自然语言叙事文本在 token 序列层面具有更丰富的局部变化。需要强调的是，gzip complexity 衡量的是静态可压缩性，不等价于数据质量，也不能直接替代训练后的 validation loss。它的作用是提供一个低成本的数据分布描述变量，用于辅助理解不同 source 的学习行为。

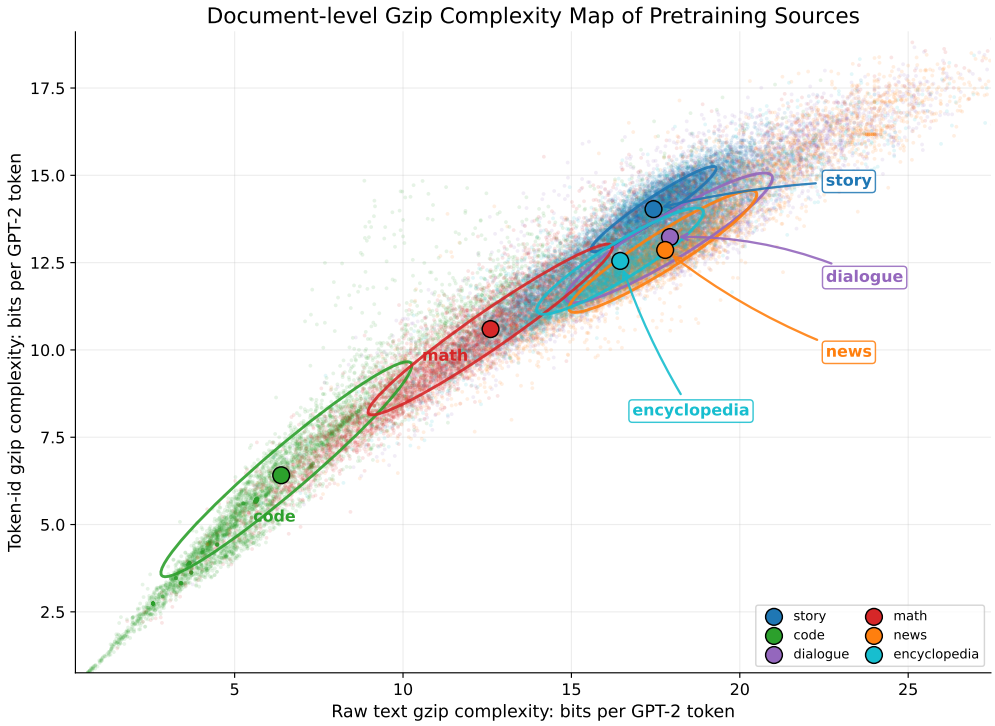


图 4: 六类训练数据的 gzip complexity map。横轴为 raw text gzip bits-per-token，纵轴为 token-id gzip bits-per-token；淡色点表示采样训练文档，黑边大点表示各 source 的文档的中位数，椭圆表示一倍协方差范围。较高的 bits-per-token 表示更低的可压缩性和更强的局部变化。

为了进一步观察不同数据分布对训练动力学的影响，本文对六类 source stream 分别进行了 fixed-step 训练实验。图 5 展示了各 source 上 quick validation loss 随累计训练计算量 C 的变化；每个子图左下角的 inset 展示同一批实验在训练 step 轴上的变化。这里使用 quick validation loss，是因为 source fixed-step 实验的 full validation checkpoints 较少，而 quick validation 每 10 step 评估一次，能够更清楚地展示训练过程中的下降形状。

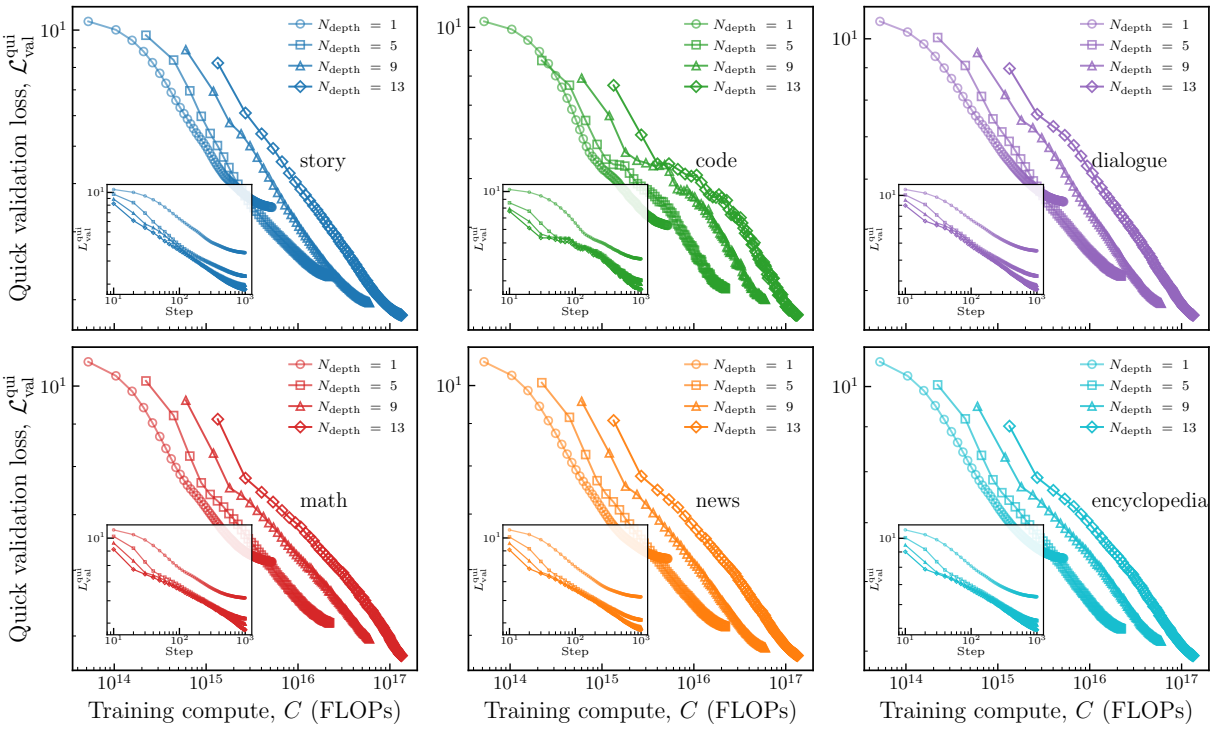


图 5: 六类 source 数据上的 fixed-step quick validation loss 曲线。每个子图对应一个数据源，主图横轴为累计训练计算量 C，左下角 inset 横轴为训练 step。该图用于展示不同数据源的训练动力学差异；正式 IsoFLOP 和 scaling exponent 拟合仍使用 full validation loss 及其尾段平均。

从 fixed-step 曲线可以看到，数据源之间的训练难度和下降速度存在明显差异。story 和 code 在短程训练中 loss 下降较快，说明这两类数据在当前模型和 tokenizer 设置下更容易被小模型快速建模。code 的这一现象与其较低的 gzip complexity 和较高的文档内重复度相一致：代码数据中存在大量重复结构、关键字和格式模板，因此模型可以较快捕捉局部规律。story 虽然 gzip complexity 较高，但文本风格相对一致，也表现出较好的可学习性。

相比之下，dialogue、news、math 和 encyclopedia 的 validation loss 整体更高，下降形状也更依赖模型规模。dialogue 和 news 涉及更丰富的主题与表达方式，局部可压缩性较低；encyclopedia 的事实密度更高，对模型记忆和上下文建模能力要求更强；math 则混合了自然语言、符号和公式结构，虽然部分模式可压缩，但整体建模难度并不低。由此可见，静态 gzip 指标和训练动力学并不是一一对应关系：gzip 能描述数据的局部可压缩性，而 validation loss 反映模型在特定训练协议下对该分布的实际学习效果。二者结合起来，能够更完整地刻画不同数据源的学习效率差异。

4.4 Source IsoFLOP：数据分布改变最优配比

为了进一步观察数据分布对 compute-optimal scaling 的影响，本文对 story、code 和 encyclopedia 三类数据源进行了 refined source IsoFLOP 实验。这三类 source 分别代表较典型的叙事文本、结构化代码文本和知识密集型百科文本，能够覆盖不同的数据复杂度与学习动力学区域。

图 6 展示了三类 source 的 fixed-FLOPs U 型曲线。每一行对应一个数据源，左列展示尾段平滑验证损失 $\mathcal{L}_{\text{tail}}$ 关于模型参数量 N 的变化，右列展示同一批实验关于训练 token 数 D 的变化。与 mixed 数据类似，在固定计算预算 C 下，不同 source 也呈现出不同程度的 U 型结构：模型过小时容量不足，模型过大时训练 token 不足，中间区域存在较优的模型–数据配比。

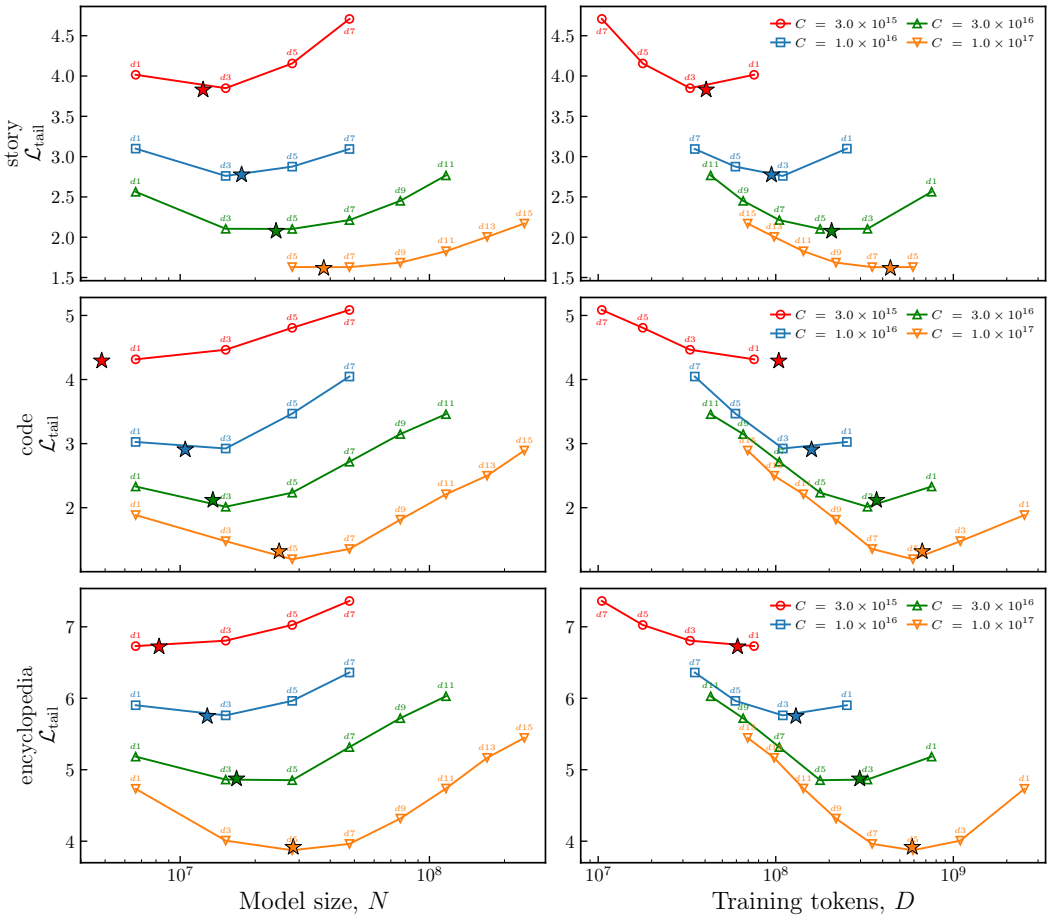


图 6: story、code 和 encyclopedia 三类数据源上的 fixed-FLOPs IsoFLOP 曲线。每一行对应一个数据源，左列为 $\mathcal{L}_{\text{tail}}$ 关于模型参数量 N 的曲线，右列为 $\mathcal{L}_{\text{tail}}$ 关于训练 token 数 D 的曲线。星号表示在 log-space 二次拟合下得到的 observed optimum。

拟合方法与 mixed IsoFLOP 部分保持一致。对每个 source 和每个固定计算预算，分别拟合

$$\mathcal{L}_{\text{tail}}(\log_{10} N) = a_N(\log_{10} N)^2 + b_N \log_{10} N + c_N, \tag{36}$$

$$\mathcal{L}_{\text{tail}}(\log_{10} D) = a_D(\log_{10} D)^2 + b_D \log_{10} D + c_D. \tag{37}$$

当二次项系数为正且顶点位于实际扫描范围内时，该预算下的最优点被记为 observed optimum。随后再对不同计算预算下的最优点做 log-log 幂律拟合，并在图 7 展示了三类 source 的 compute-optimal frontier。拟合结果为

$$\begin{aligned}
\text{story}: \quad & N^\star = 37(1)\,\text{M}\left(\frac{C}{10^{17}}\right)^{0.315(13)}, \quad D^\star = 454(13)\,\text{M}\left(\frac{C}{10^{17}}\right)^{0.684(13)}, \\
\text{code}: \quad & N^\star = 24(3)\,\text{M}\left(\frac{C}{10^{17}}\right)^{0.379(79)}, \quad D^\star = 703(84)\,\text{M}\left(\frac{C}{10^{17}}\right)^{0.621(79)}, \\
\text{encyclopedia}: \quad & N^\star = 27(1)\,\text{M}\left(\frac{C}{10^{17}}\right)^{0.344(24)}, \quad D^\star = 608(32)\,\text{M}\left(\frac{C}{10^{17}}\right)^{0.655(24)}.
\end{aligned} \tag{38}$$

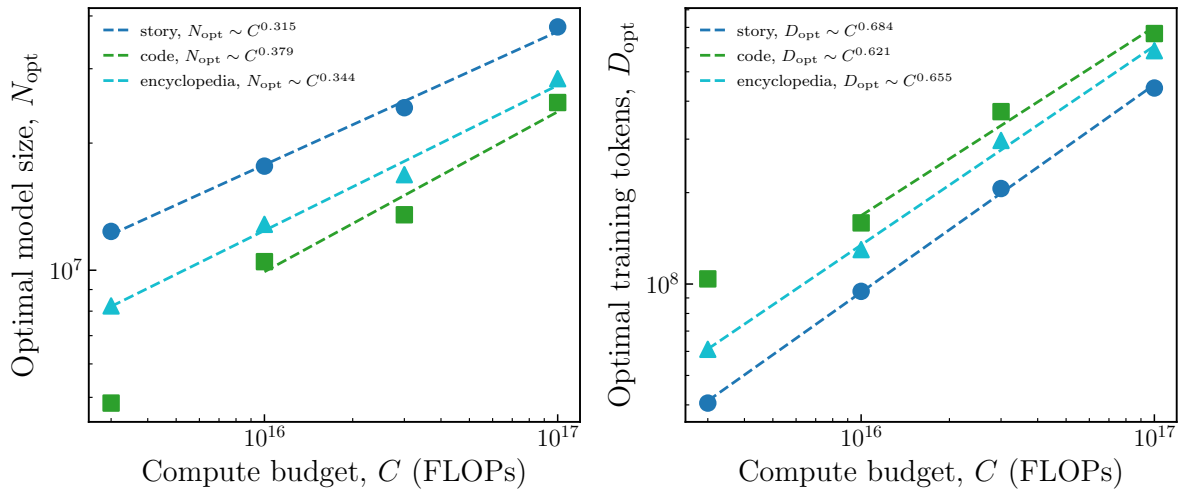


图 7: 三类数据源的 compute-optimal scaling 对比。点表示由图 6 中 IsoFLOP 曲线二次拟合得到的 optimum，虚线表示 log-log 空间中的幂律拟合。不同数据源得到的 scaling exponent 不同，说明数据分布会影响固定计算预算下的最优模型–数据配比。

从当前结果看，三类数据源的最优配比并不相同。story 的参数量指数最小、数据量指数最大，说明在本文实验范围内，随着计算预算增加，story 更偏向将新增计算量用于增加训练 token 数。code 的参数量指数相对更大，准确地说，它的最优模型规模随计算预算增长得更快，而最优 token–parameter 比例增长得更慢；这可能与代码数据中较强的结构性、长程依赖和语法约束有关。encyclopedia 位于二者之间，表现为知识密集型自然语言数据的中间状态。

为了更直观地表示这种配比差异，定义

$$R^\star(C) = \frac{D^\star(C)}{N^\star(C)}. \tag{39}$$

由前述拟合结果得到

$$\begin{aligned} R^\star_{\text{story}}(C) &= 12.4(7)\left(\frac{C}{10^{17}}\right)^{0.368(26)}, \\ R^\star_{\text{code}}(C) &= 30(7)\left(\frac{C}{10^{17}}\right)^{0.242(159)}, \\ R^\star_{\text{encyclopedia}}(C) &= 22(2)\left(\frac{C}{10^{17}}\right)^{0.312(48)}. \end{aligned} \tag{40}$$

因此，在参考计算预算 $C_0 = 10^{17}$ FLOPs 下，story、code 和 encyclopedia 的最优 token–parameter 比例分别约为 12.4、30 和 22。换言之，在该预算附近，每个模型参数对应的最优训练 token 数以 code 最高、encyclopedia 次之、story 最低。然而，当计算预算扩大 10 倍时，三者的最优比例分别扩大约 2.34、1.75 和 2.05 倍，说明 story 的最优配比向更多训练 token 转移得最快，而 code 的转移速度相对较慢。

5 总结与讨论

本文完成了一套单卡 H100 上的小规模 LLM scaling law demo。实验从数据清洗、GPT-2 tokenization、doc-level gzip complexity 统计，到 RoPE decoder-only GPT 训练、FLOPs 记录、fixed-step 学习曲线、fixed-FLOPs IsoFLOP 曲线和 compute-optimal frontier 拟合，构成了一个相对完整的

实验闭环。本文的目的不是追求大模型性能，而是在有限算力下明确模型参数量 N、训练 token 数 D、训练计算量 C 与 validation loss 之间的基本关系。

在 mixed 数据实验中，本文在 10^{16} 到 10^{18} FLOPs 范围内观察到了较清楚的 fixed-compute U 型趋势：模型过小时主要受容量限制，模型过大时则由于可训练 token 数不足而训练不充分。通过对 IsoFLOP 曲线的最低点进行拟合，得到

$$N^\star \propto C^{0.407}, \qquad D^\star \propto C^{0.593}.$$

这说明在本文的小规模实验范围和统一训练协议下，最优计算分配更偏向增加训练 token 数，但模型规模仍需随计算预算同步增长。

进一步的 source 实验表明，story、code 和 encyclopedia 得到的 compute-optimal exponent 并不相同，说明数据分布会影响固定算力下的最优模型–数据配比。结合 gzip complexity 与文档内重复度可以看到，静态可压缩性、数据重复结构和实际 validation loss 下降速度之间存在相关性，但并不是一一对应关系。因此，数据质量或数据难度不能只由 gzip 指标决定，而应结合训练动力学、验证损失和边际收益共同判断。

本文也有明显限制。首先，所有实验仍属于小规模 demo，结果不能直接外推到百亿或千亿参数模型。其次，本文主要采用单 seed 实验，尚未系统估计随机初始化和数据顺序带来的方差。第三，学习率、batch size 和优化器协议在不同模型规模之间保持统一，因此得到的是该训练协议下的 compute-optimal frontier，而不是充分调参后的全局最优 frontier。第四，本文主实验使用 $C \approx 6ND$ 作为 FLOPs 估算公式，但由于小模型中 embedding 参数占比较高，且 GPT-2 大词表下 logits projection 的计算量不可忽略，严格 FLOPs 估计仍需要进一步引入 attention- and logits-corrected compute。最后，部分高预算实验中的 training tokens 已超过 unique corpus tokens，存在数据 wrap，因此对结果和结论也有影响。

总体而言，本文展示了一个从数据分布分析到 compute-optimal scaling 拟合的完整小规模实验框架。虽然当前结果仍是 demo 级别，但它为后续扩展提供了清晰方向：引入多 seed 方差估计，使用更严格的 FLOPs 统计，区分 unique tokens 与 repeated tokens，加入 attention/logits-corrected compute，并进一步研究不同数据源在训练过程中的边际收益和饱和点。

参考文献

[1] Jared Kaplan et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. arXiv:2001.08361.

[2] Tom Henighan et al. Scaling laws for autoregressive generative modeling. *arXiv preprint arXiv:2010.14701*, 2020. arXiv:2010.14701.

[3] Jordan Hoffmann et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. arXiv:2203.15556.

[4] OpenAI. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023. arXiv:2303.08774.

[5] Nikhil Sardana et al. Beyond chinchilla-optimal: Accounting for inference in language model scaling laws. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. arXiv:2401.00448.

[6] Andrej Karpathy. nanoGPT. https://github.com/karpathy/nanoGPT, 2022. GitHub repository.

[7] Andrej Karpathy. nanochat. https://github.com/karpathy/nanochat, 2026. GitHub repository.

[8] Rohan Pandey. gzip predicts data-dependent scaling laws, 2024. arXiv:2405.16684.